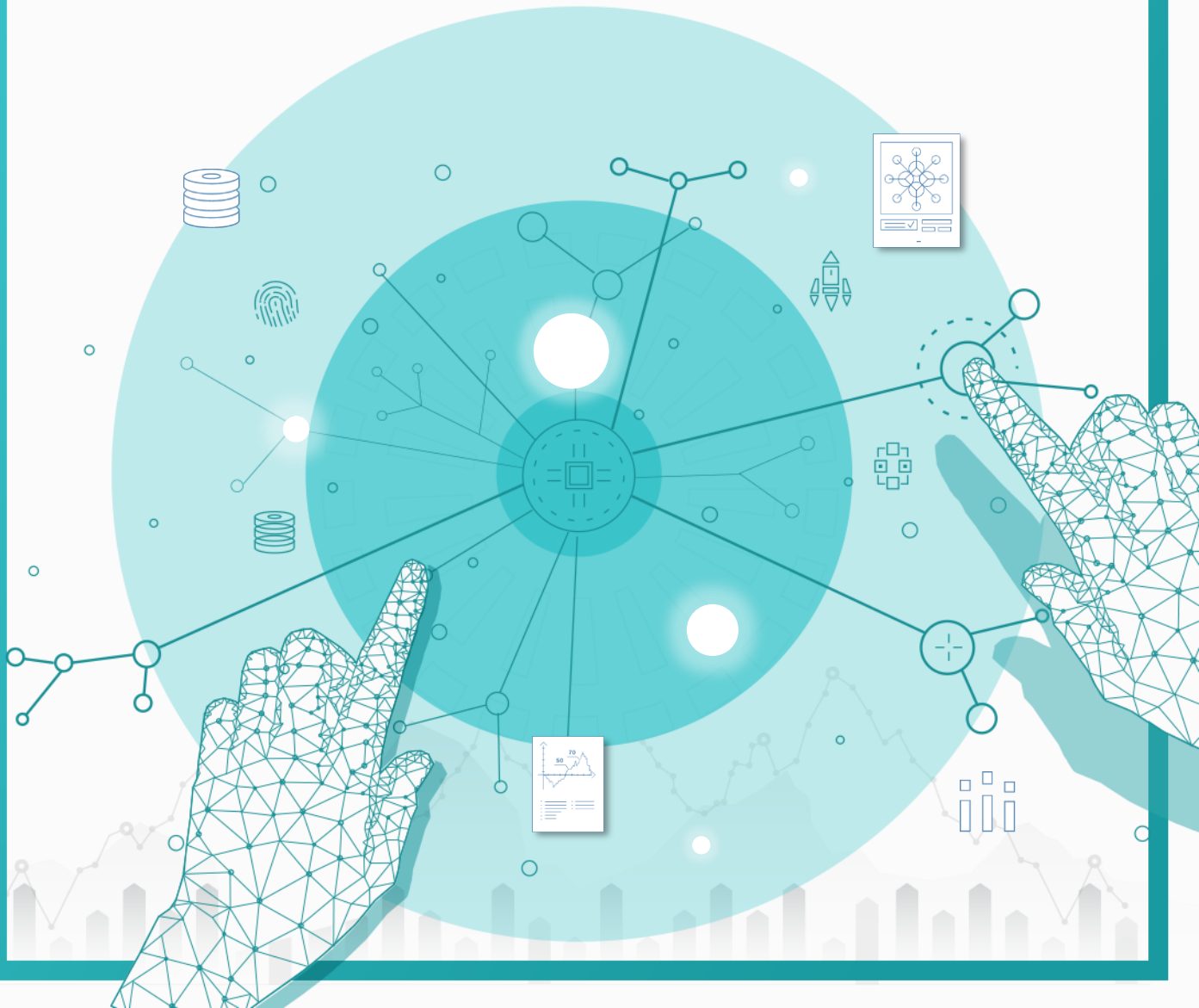
한국기술교육대학교
온라인평생교육원

「파이썬을 활용한 이미지 데이터 분석」

딥러닝 학습



딥러닝 학습

학/습/목/표

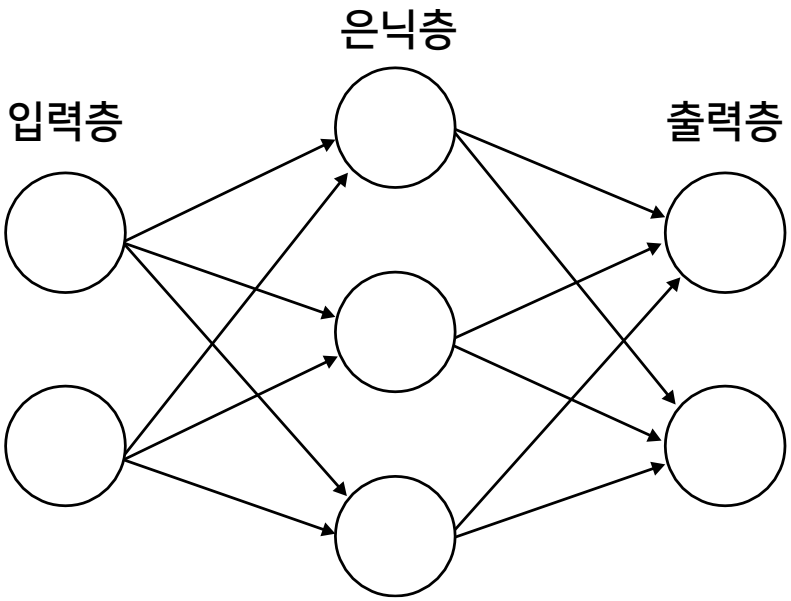
1. 활성화 함수를 학습할 수 있다.
2. 다층 신경망의 데이터 전달 과정을 학습할 수 있다.
3. 신경망 학습에 필요한 손실 함수 및 경사 하강법을 학습할 수 있다.

학/습/내/용

1. 퍼셉트론에서 신경망으로
2. 활성화 함수
3. 다층 신경망의 데이터 전달 과정
4. 신경망 학습에 필요한 손실 함수 및 경사 하강법

1. 퍼셉트론에서 신경망으로

1) 신경망의 예

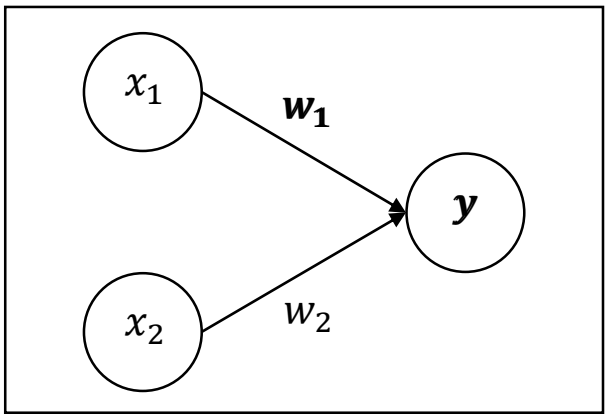


- 은닉층(1층) 뉴런: 입력층(0층), 출력층(2층)과 달리 사람 눈에 보이지 않음
- 신경망: 총 3층으로 구성
- 가중치를 갖는 층은 2개이므로 '2층 신경망'
- 신경망에서는 신호를 어떻게 전달할까?

2) 퍼셉트론과 신경망

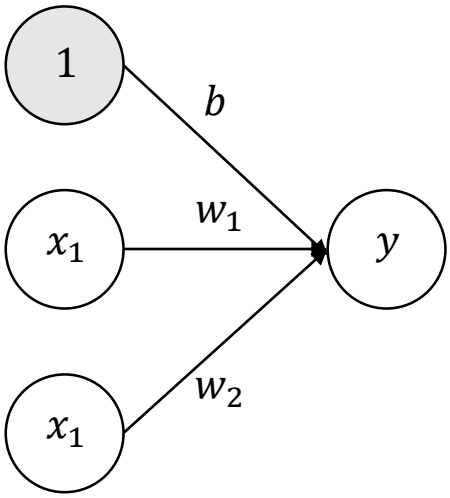
(1) 퍼셉트론

- x_1 과 x_2 신호를 입력받아 y를 출력하는 퍼셉트론



- $y = \begin{cases} 0 \ (b + w_1x_1 + \ w_2x_2 \leq 0) \\ 1 \ (b + w_1x_1 + \ w_2x_2 > 0) \end{cases}$

- 네트워크에 편향 b를 명시
  - 가중치가 " b" 이고 입력이 1인 뉴런 추가



2) 퍼셉트론과 신경망

(1) 퍼셉트론

- (A)의 네트워크에 편향 b를 명시

(A) $$y = h(b + w_1x_1 + w_2x_2)$$

(B) $$h(x) = \begin{cases} 0\ (x \leq 0) \\ 1\ (x > 0) \end{cases}$$

- 계단 함수(Step Function): 임계값을 경계로 출력이 바뀌는 함수

2. 활성화 함수

1) 활성화 함수의 등장

(1) 활성화 함수(Activation Function)

(B) $$h(x) = \begin{cases} 0 \ (x \leq 0) \\ 1 \ (x > 0) \end{cases}$$

- (B)와 같이, 입력 신호의 총합을 출력 신호로 변환하는 함수
  - 활성화 함수: 입력 신호의 총합이 활성화를 일으키는지를 정하는 역할
- (B)는 다음과 같이 (C)와 (D)로 나눌 수 있음
  - 활성화 함수: 입력 신호의 총합이 활성화를 일으키는지를 정하는 역할

(B) $$h(x) = \begin{cases} 0 \ (x \leq 0) \\ 1 \ (x > 0) \end{cases}$$

(C) $$a = b + w_1 x_1 + w_2 x_2$$

(D) $$y = h(a)$$

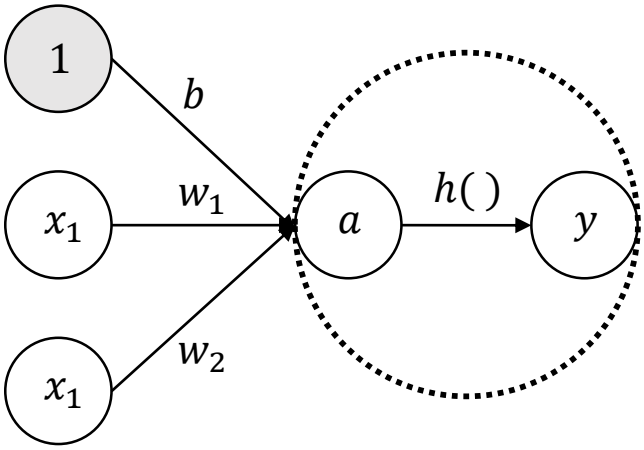
2. 활성화 함수

1) 활성화 함수의 등장

(1) 활성화 함수(Activation Function)

(D) $$y = h(a)$$

- (D)의 활성화 함수의 처리 과정



- 가중치 신호를 조합한 결과가 a라는 노드가 되고, 활성화 함수 $h(\,)$를 통과하여 y라는 노드로 변환되는 과정

▪ 단순 퍼셉트론

- 단층 네트워크에서 계단 함수를 활성화 함수로 사용한 모델

▪ 다층 퍼셉트론

- 신경망(여러 층으로 구성되고 시그모이드 함수 같은 매끈한 활성화 함수 사용)

2) 시그모이드 함수

(1) 시그모이드 함수(Sigmoid Function)

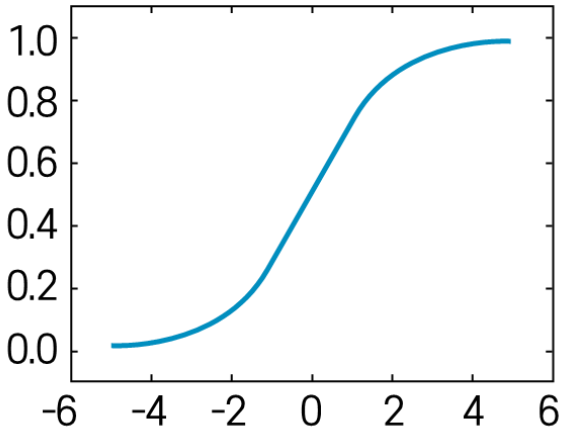
- 신경망에서 자주 이용하는 활성화 함수

$$h(x) = \frac{1}{1 + \exp(-x)}$$

$-\exp(-x)$: e^{-x}

- e: 자연상수로 2.7182..의 값을 갖는 실수

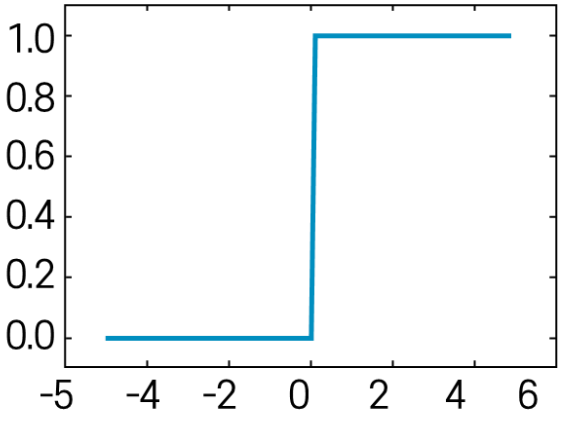
- 신경망에서는 시그모이드 함수를 활성화 함수로 이용하여 신호를 변환하고, 변환된 신호를 다음 뉴런에 전달
  - 시그모이드 함수의 그래프



2) 시그모이드 함수

(2) 계단함수

- 계단 함수: 입력이 0을 넘으면 1, 그 외에는 0을 출력하는 함수
  - 계단 함수



(3) 시그모이드 함수와 계단 함수의 차이점

- 시그모이드 함수
  - 부드러운 곡선, 입력에 따라 출력이 연속적으로 변화
  - 연속적인 실수가 흐름
- 계단 함수
  - 0(임계값)을 경계로 출력이 갑자기 변화
  - 0 또는 1이 흐름

2) 시그모이드 함수

(4) 시그모이드 함수와 계단 함수의 공통점

- 입력이 중요하면 큰 값 출력, 중요하지 않으면 작은 값 출력
- 입력이 아무리 작거나 커도 출력은 0에서 1 사이
- 비선형 함수

3) 비선형 함수

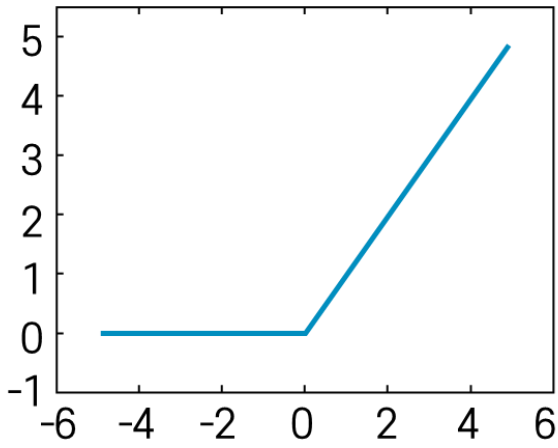
(1) 비선형 함수

- 직선 1개로는 그릴 수 없는 함수
- 신경망에서는 활성화 함수로 비선형 함수 사용
- 선형 함수를 이용하면?
  - 신경망의 층을 깊게 하는 의미가 없음
  - 선형 함수의 문제는 층을 아무리 깊게 해도 '은닉층이 없는 네트워크'로도 똑같은 기능을 할 수 있음
  - $h(x) = cx$를 활성화 함수로 사용한 3층 네트워크는 $y(x) = h(h(h(x)))$, $y(x) = c * c * c * \mathrm{x}$, 결국 $\mathrm{y(x)} = \mathrm{ax}$
  - 은닉층이 없는 네트워크로 표현 가능

4) ReLU(Rectified Linear Unit, 렐루) 함수

(1) ReLU(Rectified Linear Unit, 렐루) 함수

- 최근에는 시그모이드 함수보다 ReLU 함수를 주로 이용
- 입력이 0보다 크면 입력 그대로 출력, 0 이하라면 0을 출력



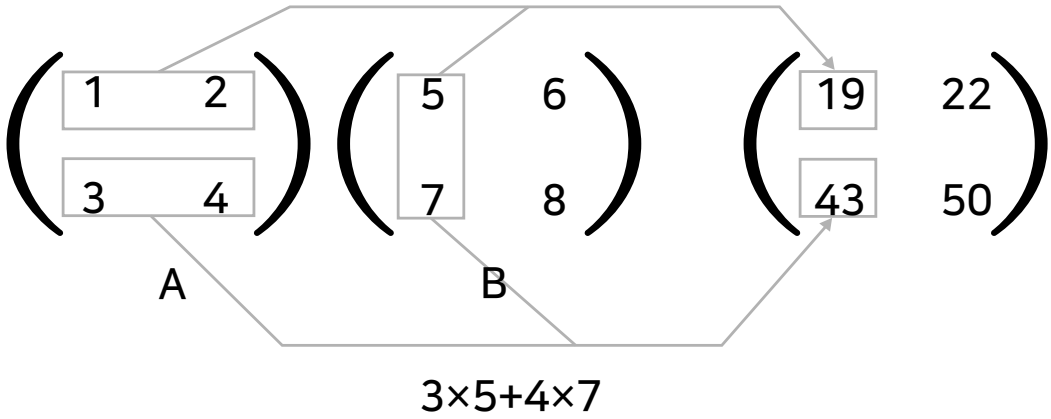
- $\text{def relu(x): return np.maximum(0, x)}$
- $h(x) = \begin{cases} x & (x > 0) \\ 0 & (x \leq 0) \end{cases}$

3. 다층 신경망의 데이터 전달 과정

1) 다차원 배열의 계산

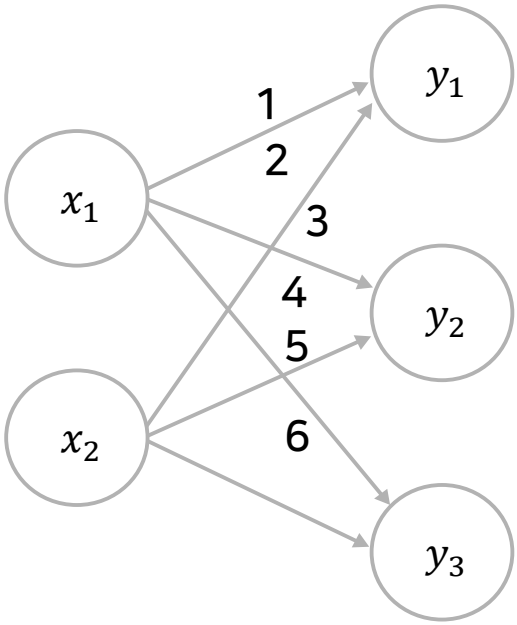
(1) 행렬의 곱

- 행렬(2차원 배열)의 곱 계산 방법



(2) 신경망에서의 행렬의 곱

- 행렬의 곱으로 신경망의 계산 수행
  - 편향과 활성화 함수 생략, 오직 가중치만 계산



$$\begin{pmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{pmatrix}$$

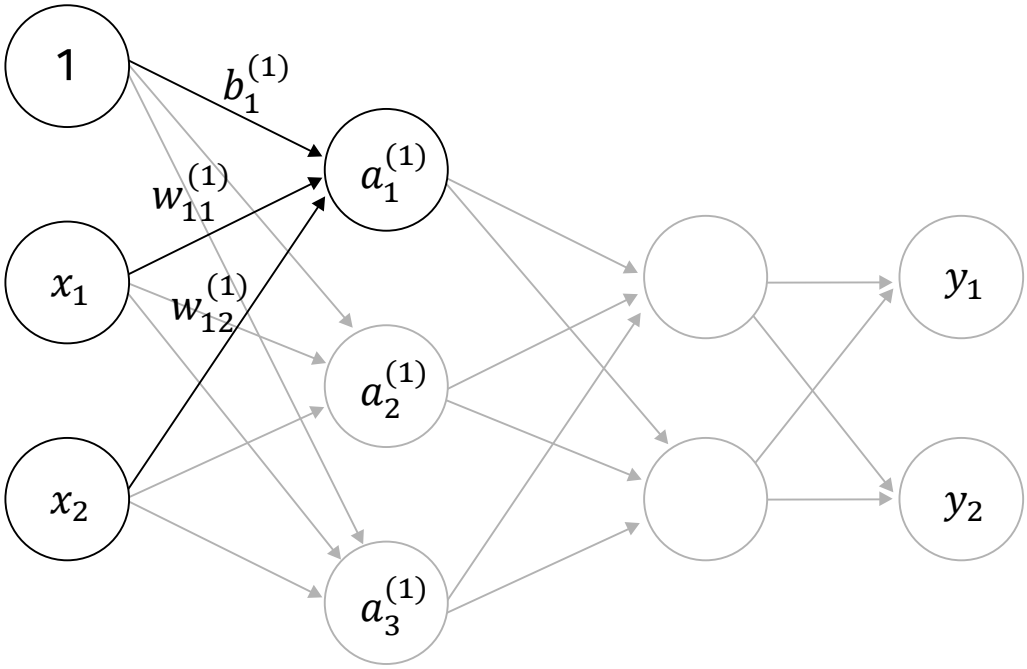
X	W	$=$	Y
2	2×3		3

3. 다층 신경망의 데이터 전달 과정

2) 다층 신경망

(1) 각 층의 신호 전달 구현하기

- 입력층에서 1층으로 신호 전달



$$a_1^{(1)} = w_{11}^{(1)}x_1 + w_{12}^{(1)}x_2 + \mathrm{b}_1^{(1)}$$

$$A^{(1)} = XW^{(1)} + B^{(1)}$$

3. 다층 신경망의 데이터 전달 과정

2) 다층 신경망

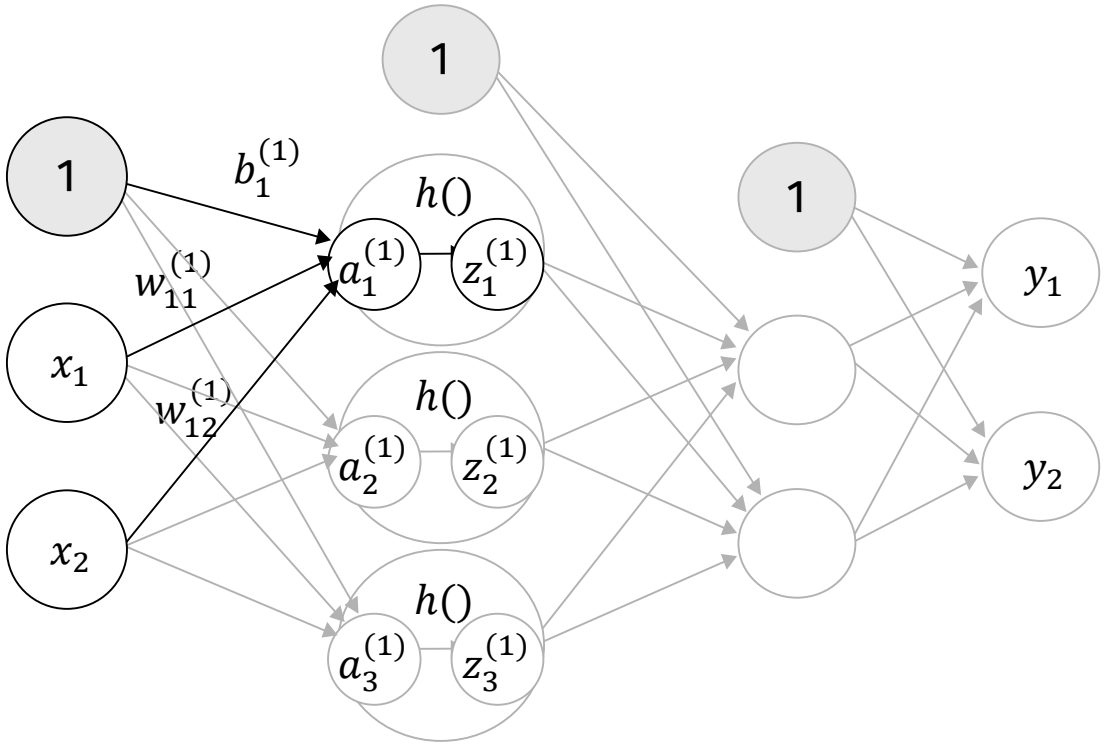
(1) 각 층의 신호 전달 구현하기

- 행렬 $A^{(1)}, X, B^{(1)}, W^{(1)}$은 다음과 같음

$$A^{(1)} = \left(a_1^{(1)}, a_2^{(1)}, a_3^{(1)}\right),\ X = (x_1, x_2),\ B^{(1)} = (b_1^{(1)}, b_2^{(1)}, b_3^{(1)})$$

$$w^{(1)} = \begin{pmatrix} w_{11}^{(1)} & w_{21}^{(1)} & w_{31}^{(1)} \\ w_{12}^{(1)} & w_{22}^{(1)} & w_{32}^{(1)} \end{pmatrix}$$

- 입력층에서 1층으로의 신호 전달

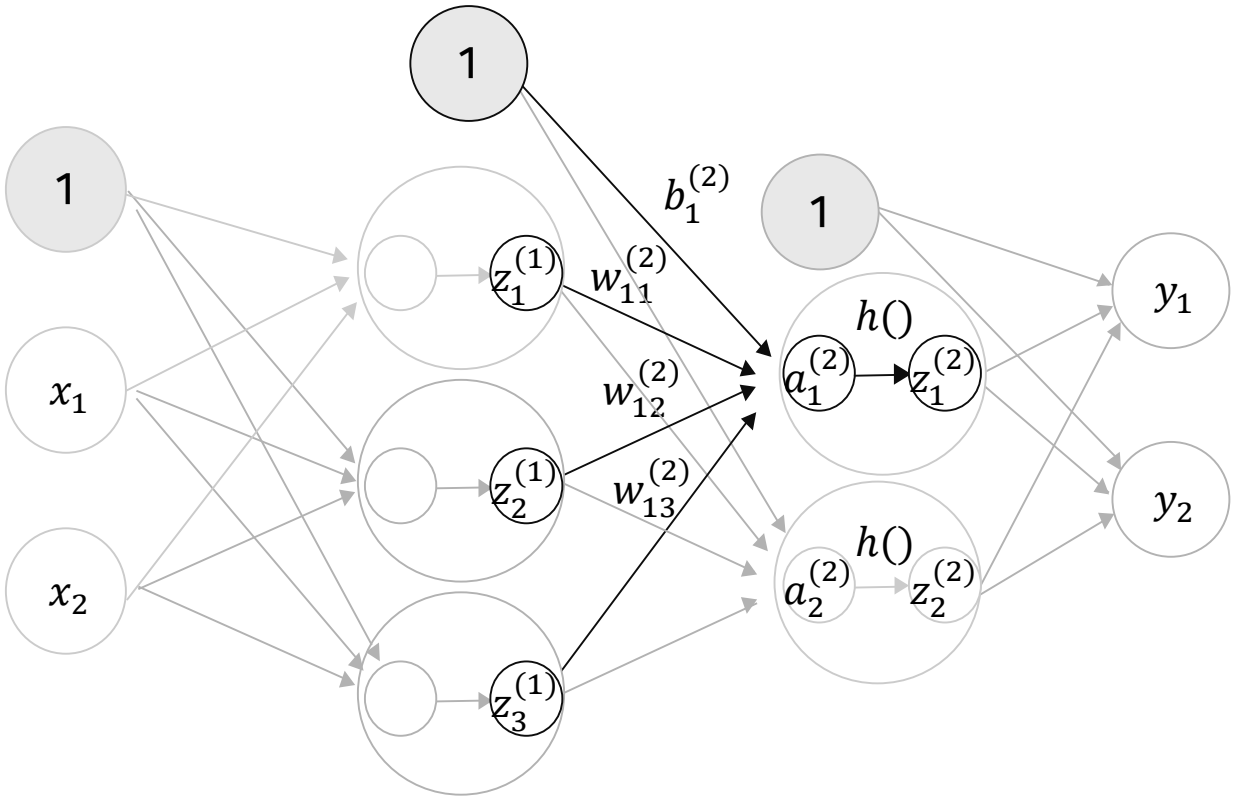


3. 다층 신경망의 데이터 전달 과정

2) 다층 신경망

(1) 각 층의 신호 전달 구현하기

- 1층에서 2층으로의 신호 전달

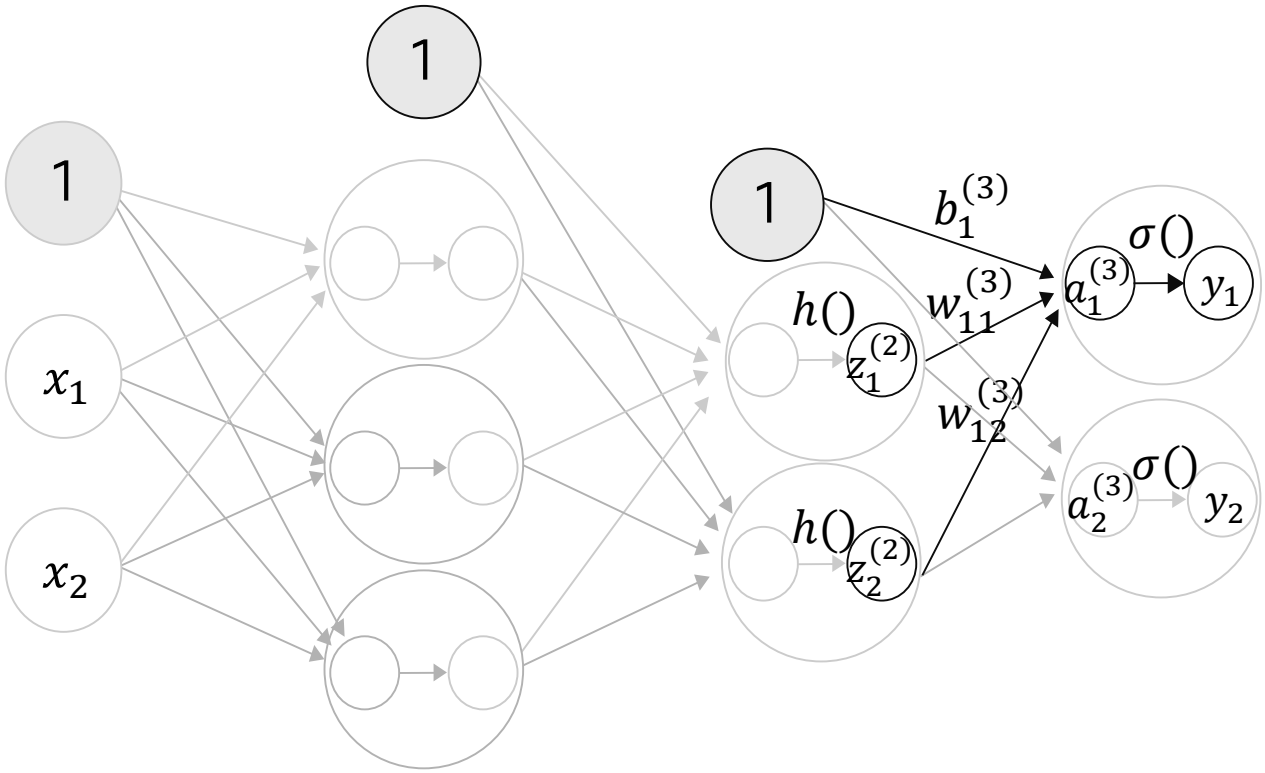


3. 다층 신경망의 데이터 전달 과정

2) 다층 신경망

(1) 각 층의 신호 전달 구현하기

- 2층에서 출력층으로의 신호 전달



- 활성화 함수만 지금까지의 은닉층과 다름

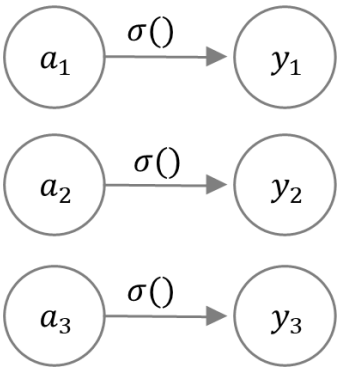
3) 출력층

(1) 개요

- 신경망은 분류와 회귀 모두 이용 가능
  - 분류(Classification): 데이터가 어느 클래스에 속하느냐는 문제
    (예) 사진 속 인물의 성별 분류
  - 회귀(Regression): 입력 데이터에서 (연속적인) 수치 예측
    (예) 사진 속 인물의 몸무게 예측
- 둘 중 어떤 문제이냐에 따라 출력층에서 사용하는 활성화 함수가 달라짐
  - 분류(Classification): 소프트맥스(Softmax) 함수
  - 회귀(Regression): 항등 함수

(2) 항등 함수와 소프트맥스(Softmax) 함수 구현하기

- 항등 함수(Identity Function): 입력과 출력이 같음
  - 항등 함수



3. 다층 신경망의 데이터 전달 과정

3) 출력층

(3) 소프트맥스(Softmax) 함수의 특징

- 소프트맥스(Softmax) 함수 출력
  - 0에서 1.0 사이의 실수
- 소프트맥스(Softmax) 함수 출력의 총합: 1
- 소프트맥스(Softmax) 함수의 출력을 '확률'로 해석 가능

```
a = np.array([0.3,2.9,4.0])
y = softmax(a)
print(y)
np.sum(y)
```

결과

```
[0.01821127 0.24519181 0.73659691]
1.0
```

- y[0]의 확률: 0.018(1.8%)
- y[1]의 확률: 0.245(24.5%)
- y[2]의 확률: 0.737(73.7%)

→ 결론: "2번 원소의 확률이 가장 높으니, 답은 2번 클래스!" 또는 "74% 확률로 2번 클래스"

3. 다층 신경망의 데이터 전달 과정

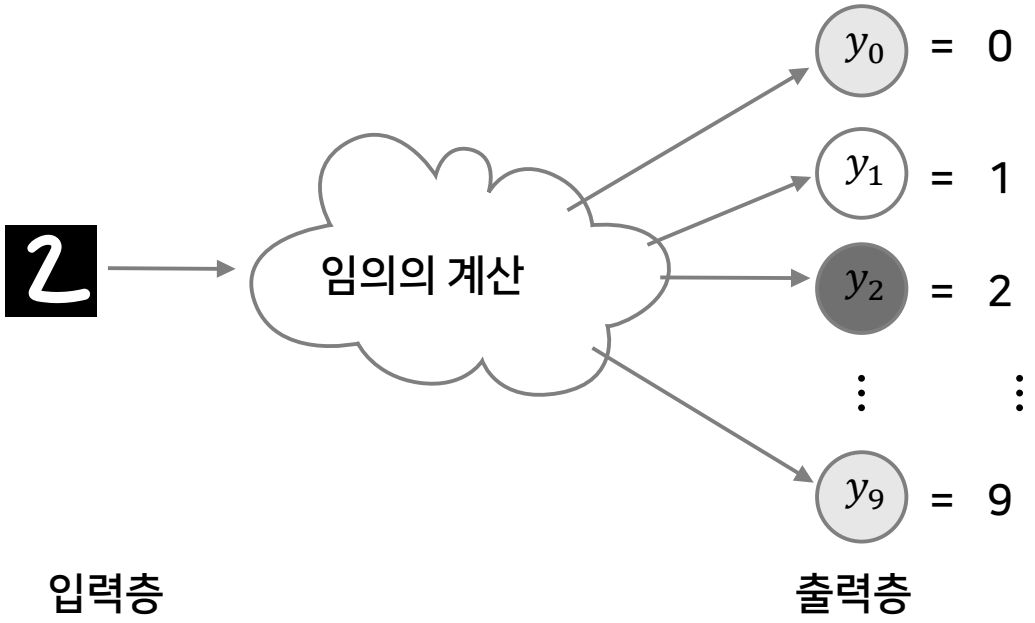
3) 출력층

(3) 소프트맥스(Softmax) 함수의 특징

- 소프트맥스(Softmax) 함수를 적용해도 각 원소의 대소 관계는 변하지 않음
- 신경망으로 분류 시, 출력층의 소프트맥스(Softmax) 함수를 생략해도 무방(자원 낭비 감소 효과)

(4) 출력층의 뉴런 수 정하기

- 해결하려는 문제에 맞게 적절히 설정
- 분류문제에서는 분류하고 싶은 클래스 수로 설정하는 것이 일반



4. 신경망 학습에 필요한 손실 함수 및 경사 하강법

1) 신경망 학습

(1) 데이터에서 학습

- 신경망의 특징
  - 데이터를 보고 학습할 수 있다는 점
  - 가중치 매개변수의 값을 데이터를 보고 자동으로 결정
- 만약 수천, 수만, 수억에 이르는 모든 매개변수를 수작업으로 결정해야 한다고 상상

(2) 데이터 주도 학습

- 기계학습의 중심에는 데이터 존재
  - 데이터에서 답을 찾고, 패턴을 발견하고, 이야기를 만듦
  - 데이터가 이끄는 접근 방식 덕분에 사람 중심 접근에서 벗어날 수 있음
- 기계학습에서는 사람의 개입을 최소화하여 수집한 데이터에서 패턴 파악
- 신경망과 딥러닝
  - 기존 기계학습보다 사람의 개입을 더욱 배제할 수 있게 해주는 중요한 특성 보유

4. 신경망 학습에 필요한 손실 함수 및 경사 하강법

1) 신경망 학습

(2) 데이터 주도 학습

- 이미지에서 '5'라는 숫자를 인식하는 프로그램을 구현한다고 가정
  - 손글씨 숫자 '5'의 예: 사람마다 자신만의 필체가 있음

(3) 데이터 주도 학습(cont.)

- '5'를 제대로 분류하는 프로그램을 직접 고안해 설계하기란 의외로 어려운 문제
- '5'를 인식하는 알고리즘을 밑바닥부터 설계하는 대신, 주어진 데이터를 활용해서 해결
  - 이미지에서 특징(Feature) 추출하여, 그 특징의 패턴을 기계학습 기술로 학습하는 방법
    - 특징: 입력 데이터(이미지)에서 본질적인 데이터를 정확하게 추출할 수 있도록 설계된 변환기

4. 신경망 학습에 필요한 손실 함수 및 경사 하강법

1) 신경망 학습

(3) 데이터 주도 학습(cont.)

- 이미지의 특징: 보통 벡터(Vector)로 기술, 컴퓨터 비전 분야에서는 SIFT, SURF, HOG 등의 특징을 많이 사용
  - 이미지 데이터를 벡터로 변환하고, 변환된 벡터를 가지고 지도 학습 방식의 대표적인 분류 기법인 SVM, KNN 등으로 학습
    - 기계학습에서는 데이터에서 규칙을 찾아내는 역할을 기계가 담당
    - 이미지를 벡터로 변환할 때 사용하는 특징은 여전히 사람이 설계
- 규칙을 사람이 만드는 방식에서 기계가 데이터로부터 배우는 방식으로의 패러다임 전환

5 → 사람이 생각한 알고리즘 → 결과

5 → 사람이 생각한 특징 (SIFT, HOG 등) → 기계학습 (SVM, KNN 등) → 결과

5 → 신경망(딥러닝) → 결과

4. 신경망 학습에 필요한 손실 함수 및 경사 하강법

1) 신경망 학습

(3) 데이터 주도 학습(cont.)

- 특징을 사람이 설계했지만, 신경망은 주요 특징까지도 기계가 스스로 학습
- 딥러닝 = 종단간 기계학습(End-to-End Machine Learning)
  - 의미: 데이터(입력)에서 목표한 결과(출력)를 사람의 개입 없이 얻음
- 신경망의 이점

  : 모든 문제를 같은 맥락에서 풀 수 있음
  - '5'를 인식, '개'를 인식, '사람의 얼굴'을 인식하는 문제 등 세부사항과 관계없이 신경망은 주어진 데이터를 온전히 학습하고, 주어진 문제의 패턴을 발견하려고 시도

(4) 훈련 데이터와 시험 데이터

- 기계학습 문제는 훈련 데이터(Training Data)와 시험 데이터(Test Data)로 나눠 학습과 시험 수행
  - 훈련 데이터만 사용하여 학습하면서 최적의 매개변수를 찾음
    → 시험 데이터를 사용하여 앞서 훈련한 모델의 실력 평가

1) 신경망 학습

(4) 훈련 데이터와 시험 데이터

- 왜 훈련 데이터와 시험 데이터를 나눌까?
  - 우리가 원하는 것은 범용적으로 사용할 수 있는 모델이기 때문
  - 훈련된 모델의 범용 능력(Generalization Performance): 아직 보지 못한 데이터(훈련 데이터 미포함)로도 문제를 올바르게 풀어내는 능력
    - 기계학습의 최종 목표: 범용 능력 획득
- 과적합(과대적합, 오버피팅, Overfitting)
  - 특정 데이터 세트에만 지나치게 최적화된 상태
  - 과적합 문제를 피하는 것은 기계학습의 중요한 과제의 하나
  - 과적합을 다르게 설명해보자면, 한쪽 이야기만 너무 들어서 편견이 생겨버린 상태라고 이해해도 좋음

2) 손실 함수

(1) 손실 함수

- 자신의 행복을 행복 지표(지수)를 기준으로 나타내고, 이를 기준으로 인생을 살아가는 사람이 있다면?
  - 신경망 학습에서는 현재의 상태를 하나의 지표로 표현
  - 그 지표를 가장 좋게 만들어주는 가중치 매개변수의 값 탐색
- 신경망 학습에서 사용하는 지표는 손실 함수(Loss Function, 비용 함수, Cost Function)
  - 손실 함수는 신경망 성능의 '나쁨'을 나타내는 지표
  - '나쁨' 을 최소화하는 것이 목표
- 신경망은 손실 함수를 기준으로 최적의 매개변수 값 탐색
  - 마치 행복 지표를 가진 사람이 그 지표를 근거로 최적의 인생을 탐색하듯이 해야 함
- 대표적인 손실 함수
  - 오차제곱합과 교차 엔트로피 오차

4. 신경망 학습에 필요한 손실 함수 및 경사 하강법

2) 손실 함수

(1) 손실 함수

- 오차제곱합(Sum of Squares for Error; SSE)의 수식

$$E = \frac{1}{2}\sum_{k}(y_k - t_k)^2$$

- y_k: 신경망의 출력(추정값; 예측값)
- t_k: 정답 레이블(실제값; 목표값)
- k: 데이터의 차원 수

- 교차 엔트로피 오차(Cross Entropy Error, CEE)의 수식
- 교차 엔트로피 오차의 수식

$$E = -\sum_{k} t_k \log y_k$$

- Log: 밑이 e인 자연로그($log\ _e$)
- t_k: 정답에 해당하는 원소 1, 나머지 0

2) 손실 함수

(1) 손실 함수

- 교차 엔트로피 오차의 수식

$$E = -\sum_{k} t_k \log y_k$$

- 실질적으로 정답일 때의 추정값(t_k가 1일 때의 t_k) 자연로그 계산
- 정답이 아닌 나머지 모두는 t_k 가 0이므로 결과에 영향을 주지 않음
- 정답 '2', 신경망 출력 0.6, 교차 엔트로피 오차:

$$-\log 0.6 = 0.51$$

- 정답 '2', 신경망 출력 0.1, 교차 엔트로피 오차:

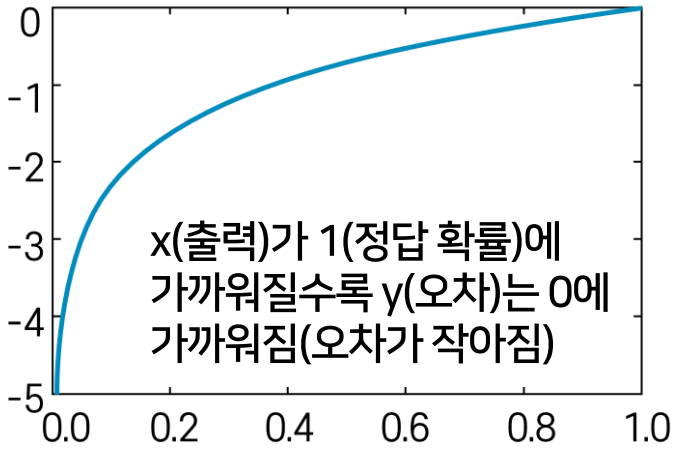
$$-\log 0.1 = 2.30$$

→ 교차 엔트로피는 정답일 때의 출력이 전체 값 결정

2) 손실 함수

(1) 손실 함수

- 자연로그 y=log$x$ 의 그래프



(2) 미니배치(Minibatch) 학습

- 모든 훈련 데이터에 대한 손실 함수의 합을 구하는 방법
  - 훈련 데이터가 100개가 있으면 그로부터 계산한 100개의 손실 함수 값들의 합을 지표로 삼는 것

4. 신경망 학습에 필요한 손실 함수 및 경사 하강법

2) 손실 함수

(2) 미니배치(Minibatch) 학습

- 교차 엔트로피 오차

$$E = -\frac{1}{N}\sum_{n}\sum_{k} t_{nk} \log y_{nk}$$

- N: 데이터의 개수
- t_{nk} : n번째 데이터의 k번째 값

  - 마지막에 N으로 나누어 정규화, '평균 손실 함수'를 구함
  - 평균을 구해 사용하면 훈련 데이터 개수와 관계없이 언제든 통일된 지표를 얻을 수 있음

- MNIST 데이터 세트의 훈련 데이터: 60,000개

  - 이 많은 데이터를 대상으로 일일이 손실 함수를 계산하기는 비현실적
  - 이런 경우 데이터 일부를 추려, 전체의 근사치로 이용 가능

2) 손실 함수

(2) 미니배치(Minibatch) 학습

- 신경망 학습에서도 훈련 데이터에서 일부(미니배치)만 골라 학습 수행
  - 가령 60,000장의 훈련 데이터에서 100장을 무작위로 뽑아 그 100장만 학습하는 것
  - 텔레비전의 시청률도 모든 세대의 텔레비전이 아니라 선택된 일부 가구의 텔레비전만을 대상으로 함

(3) 손실 함수 사용 이유

- 왜 굳이 손실 함수를 사용해야 하는 걸까?
  - 숫자 인식의 경우 높은 '정확도'를 끌어내는 매개변수를 찾는 것

(4) 손실 함수 설정 이유

- '정확도'라는 지표를 놔두고 '손실 함수의 값'이라는 우회적인 방법을 택한 이유는?
  - 신경망 학습에서의 '미분'의 역할에 주목
    - 신경망 학습에서 최적의 매개변수를 탐색할 때 손실 함수의 값을 가능한 한 작게 하는 매개변수 탐색
    - 매개변수의 미분(기울기) 계산, 그 미분 값을 단서로 매개변수의 값을 서서히 갱신하는 과정 반복

2) 손실 함수

(4) 손실 함수 설정 이유

- 가중치 매개변수에 대한 손실 함수 값의 분: '가중치의 값을 아주 조금 변화시켰을 때, 손실 함수의 값이 어떻게 변하나'
  - 음수: 가중치를 양의 방향으로 변화시켜 손실 함수의 값 감소 가능
  - 양수: 가중치를 음의 방향으로 변화시켜 손실 함수의 값 감소 가능

  (주의) 미분 값 0: 가중치를 어느 방향으로 변화시켜도 손실 함수의 값이 줄어들지 않음(갱신 정지)

- '정확도' 지표는 매개변수에 대한 미분이 대부분의 장소에서 0이 되기 때문에 신경망 학습에 부적합

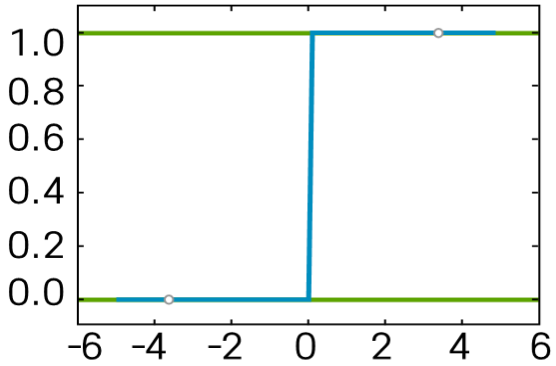
(5) 손실 함수 설정 이유(cont.)

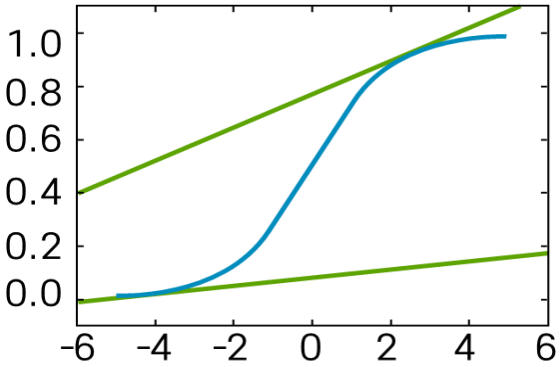
- 정확도를 지표로 삼으면 반응이 있더라도 그 값이 불연속적으로 갑자기 변화
  - 계단 함수를 활성화 함수로 사용하지 않는 이유와 연관
- 손실 함수를 지표로 삼으면, 매개변수의 값 변화에 반응하여 연속적으로 변화

2) 손실 함수

(5) 손실 함수 설정 이유(cont.)

- 계단 함수와 시그모이드 함수





- 계단 함수: 미분(기울기)은 대부분의 장소에서 0
- 시그모이드 함수: 미분은 어느 장소에서 0이 되지 않음

3) 수치 미분

(1) 미분

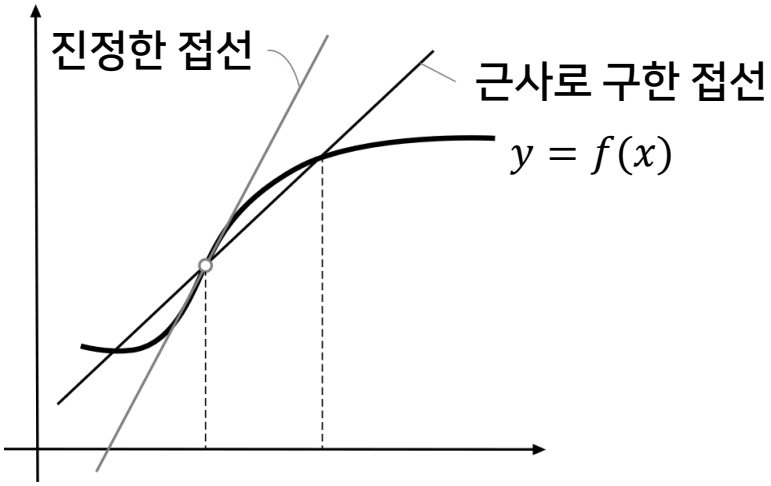
- 가정: 어떤 마라톤 선수가 처음부터 10분 동안 2km를 달림
  - 속도: 간단히 2 / 10 = 0.2 [km/분] 계산 가능
  - 1분에 0.2km 만큼의 속도(변화)로 뛰었다고 해석 가능
  - 정확하게는 10분 동안의 평균 속도
- 특정 순간의 변화량
  - 10분이라는(직전 1분, 1초, 0.1초 거리, ... 식) 시간을 가능한 줄여 한 순간의 변화량 계산
  - $\frac{df(x)}{dx} = \lim_{h \to 0} \frac{f(x+h)-f(x)}{h}$

    - 시간의 작은 변화(시간을 뜻하는 h를 한없이 0에 가깝게 한다): $\lim_{h \to 0} *$

3) 수치 미분

(2) 미분(cont.)

- 해석적 미분(접선)과 수치 미분(근사로 구한 접선)의 값은 다름



- 수치 미분에는 오차 포함
- 오차 감소를 위해 $(x+h)$와 $(x-h)$일 때의 함수 f 의 차분을 계산하는 방법 사용 가능
  - x를 중심으로 그 전후의 차분을 계산한다: '중심 차분', '중앙 차분'

4. 신경망 학습에 필요한 손실 함수 및 경사 하강법

4) 기울기

(1) 경사법(경사 하강법: Gradient Descent Method)

- 신경망: 학습을 통해 최적의 매개변수(가중치와 편향)를 찾아냄

  ↓

  손실 함수의 값이 최소가 될 때의 매개변수 값

  - 매개변수 공간은 매우 광대하여 어디가 최솟값인지 짐작 불가

- 기울기를 이용해 함수의 최솟값(가능한 작은 값)을 찾는 것
  - 각 지점에서 함수의 값을 낮추는 방안 제시 지표(단서): 기울기
    - 기울기가 가리키는 곳에 실제로 함수의 최솟값이 있는지, 그 길이 정말로 나아갈 방향이 맞는지는 보장 불가
  - 함수가 극솟값, 최솟값, 또는 안장점(Saddle Point)이 되는 곳에서 기울기: 0
    - 극솟값: 국소적인 최솟값, 즉 한정된 범위에서의 최솟값
    - 안장점: 어느 방향에서 보면 극댓값, 다른 방향에서 보면 극솟값

- 기울기가 0인 장소를 찾지만, 그곳이 꼭 최솟값이라고는 할 수 없음
  - 복잡한 모양의 함수라면 평평한 곳으로 파고들면서 고원(Plateau)이라 하는 학습 정체기에 빠질 수 있음

4. 신경망 학습에 필요한 손실 함수 및 경사 하강법

4) 기울기

(1) 경사법(경사 하강법: Gradient Descent Method)

- 기울어진 방향이 꼭 최솟값을 가리키는 것은 아니나, 그 방향으로 가야 함수의 값을 줄일 수 있음
- 경사법은 현 위치에서 기울기를 구한 뒤 기울어진 방향으로 일정 거리만큼 이동하고, 이를 반복
- 경사법 수식

$$x_0 = x_0 - \eta \frac{\partial f}{\partial x_0}$$

$$x_1 = x_1 - \eta \frac{\partial f}{\partial x_1}$$

- η(eta, 에타): 갱신하는 양
- 신경망 학습: 학습률(Learning Rate)

- 한 번의 학습으로 얼마만큼 학습해야 할지, 즉 매개변수 값을 얼마나 갱신할지 결정
- 학습률: 0.01이나 0.001 등 미리 특정 값으로 설정
- 학습률이 너무 크거나 작으면 '좋은 장소'를 찾아갈 수 없음
- 1회에 해당하는 갱신이며, 이 단계 반복
- 변수의 수가 더 늘어나도 같은 식(각 변수의 편미분 값)으로 갱신

4. 신경망 학습에 필요한 손실 함수 및 경사 하강법

5) 학습 알고리즘

(1) 신경망 학습의 절차

1단계	2단계	3단계	4단계
미니배치	기울기 산출	매개변수 갱신	반복

- 1단계 미니배치
  - 훈련 데이터 중 일부를 무작위로 가져옴
  - 선별한 데이터(미니배치)의 손실 함수 값 감소
- 2단계 기울기 산출
  - 각 가중치 매개변수의 기울기를 구함
  - 기울기: 손실 함수의 값을 가장 작게 하는 방향 제시
- 3단계 매개변수 갱신
  - 가중치 매개변수를 기울기 방향으로 아주 조금 갱신
- 4단계 반복
  - 1 ~ 3단계 반복

4. 신경망 학습에 필요한 손실 함수 및 경사 하강법

5) 학습 알고리즘

(1) 신경망 학습의 절차

- 확률적 경사 하강법(Stochastic Gradient Descent; SGD)
  - 미니배치로 무작위로 골라낸 데이터에 대해 수행

(2) 시험 데이터로 평가하기

- 손실 함수 값
  - 훈련 데이터의 미니배치에 대한 손실 함수의 값
  - 훈련 데이터의 손실 함수 값 감소: 신경망이 잘 학습함
    - 단, 이 결과만으로는 다른 데이터셋에도 비슷한 성능이 나올지 불확실

- 신경망 학습에서는 훈련에 사용되지 않은 데이터를 올바르게 인식하는지 확인
  - 과적합(Overfitting)을 일으키지 않는지 확인
  - '과적합이 일어났다': 훈련 데이터에 포함된 이미지만 제대로 구분하고, 그렇지 않은 이미지는 식별할 수 없다는 뜻

5) 학습 알고리즘

(2) 시험 데이터로 평가하기

- 신경망의 목표: 범용 능력(일반화 성능)을 갖추는 것
  - 훈련 데이터에 포함되지 않은 데이터를 이용해 평가
- 학습 도중 정기적으로 훈련 데이터와 시험 데이터를 대상으로 정확도 기록
  - 1에폭 당 훈련 데이터와 시험 데이터에 대한 정확도 기록
  - 에폭(Epoch): 하나의 단위, 1에폭은 학습에서 훈련 데이터를 모두 소진했을 때의 횟수

(예) 훈련 데이터 10,000개를 100개의 미니배치로 학습할 경우, 확률적 경사 하강법을 100회 반복하면 모든 훈련 데이터 소진, 이 경우 100회가 1에폭에 해당